

Luisa María González Álvarez

COD. 100159607

Séptimo Semestre

Facultad de Ingeniería, Ingeniería de Software

Corporación Universitaria Iberoamericana

Calidad de software

William Ruiz

03 octubre 2026

INDICE

Herramientas de pruebas de software para equipos de control de calidad	3
Selenium – Automatización de pruebas web.....	3
Postman – Pruebas de API	3
Apache JMeter – Pruebas de carga y rendimiento	4
Cypress – Pruebas end-to-end (E2E) y de componentes	4
Allure – Gestión de pruebas y reportería.....	5
SonarQube – Análisis estático de código	5
Análisis de características y funcionalidades	6
Selenium:.....	6
Postman:	6
Apache JMeter:	7
Cypress:	7
Allure:	7
SonarQube:	7
Integración en el proceso de desarrollo:	8
Ventajas y desventajas.....	9
Selenium:.....	9
Postman:	9
JMeter:	9
Cypress:	10
Allure:	10
SonarQube:	10
Caso de estudio Boeing Starliner (2019).....	10
Capturas del proceso y uso de la herramienta Postman	11
Conclusiones:	17
Aprendizajes:	17
Recomendaciones:.....	18
Referencias bibliográficas:	18

Herramientas de pruebas de software para equipos de control de calidad

Selenium – Automatización de pruebas web

Herramienta de código abierto que permite controlar navegadores (Chrome, Firefox, Edge, Safari) mediante código para simular las acciones de un usuario, como hacer clic, escribir en formularios o navegar entre páginas. Es compatible con lenguajes como Java, Python, C#, JavaScript y Ruby, lo que permite adaptarla al stack del equipo. Se usa principalmente en pruebas funcionales y de regresión.

Justificación: La seleccioné porque, durante la investigación, fue la herramienta que más aparecía como referencia en automatización de pruebas web y en ofertas laborales del área de QA. Al ser gratuita y compatible con varios lenguajes, me permite aprenderla usando un lenguaje que ya conozco, y entenderla me da bases para comprender otras herramientas de automatización que se construyen sobre ella.

Postman – Pruebas de API

Plataforma para diseñar, enviar y validar peticiones a servicios web (REST, GraphQL, SOAP). Permite organizar las peticiones en colecciones, escribir validaciones automáticas sobre las respuestas y ejecutarlas desde la línea de comandos con Newman para integrarlas en pipelines de CI/CD. Tiene un plan gratuito con límites y planes de pago para equipos.

Justificación: La escogí porque la mayoría de aplicaciones actuales se comunican a través de APIs, y probarlas es una necesidad en casi cualquier proyecto. Postman tiene una

interfaz visual que permite enviar peticiones y revisar respuestas sin necesidad de programar desde el inicio, lo que la hace una buena puerta de entrada para alguien que está empezando en pruebas de backend.

Apache JMeter – Pruebas de carga y rendimiento

Herramienta de código abierto basada en Java que simula múltiples usuarios concurrentes enviando solicitudes a una aplicación o servidor. Mide tiempos de respuesta, rendimiento (throughput) y tasa de errores bajo distintos niveles de carga. Además de HTTP, soporta otros protocolos como bases de datos (JDBC) y FTP.

Justificación: La incluí porque quería cubrir también las pruebas no funcionales, ya que una aplicación no solo debe funcionar bien, sino también soportar a muchos usuarios al mismo tiempo. Es gratuita, cuenta con mucha documentación y tutoriales, y permite crear pruebas desde una interfaz gráfica, lo que facilita su aprendizaje.

Cypress – Pruebas end-to-end (E2E) y de componentes

Framework para probar aplicaciones web modernas que se ejecuta directamente dentro del navegador, lo que le da pruebas rápidas, esperas automáticas y la posibilidad de ver paso a paso qué pasó en cada prueba. Su principal limitación es que solo trabaja con JavaScript y TypeScript. Es de código abierto, con un servicio en la nube (Cypress Cloud) de pago para reportes y ejecución en paralelo.

Justificación: La seleccioné para comparar dos enfoques distintos de automatización web junto con Selenium. Según la documentación y las fuentes consultadas, Cypress tiene

una configuración más sencilla y permite ver la ejecución de las pruebas en tiempo real, lo que resulta útil para un estudiante que necesita entender qué hace cada paso de una prueba.

Allure – Gestión de pruebas y reportería

Allure tiene dos versiones: Allure Report, de código abierto, que genera reportes visuales a partir de los resultados de frameworks como Selenium, Cypress, JUnit o pytest; y Allure TestOps, plataforma comercial que centraliza pruebas manuales y automatizadas, permite hacer seguimiento histórico y facilita la colaboración del equipo. No ejecuta pruebas por sí misma, sino que consolida y analiza los resultados de otras herramientas.

Justificación: La escogí porque, al investigar, encontré que ejecutar pruebas no es suficiente: también hay que comunicar los resultados de forma clara. Allure permite generar reportes visuales a partir de herramientas como Selenium y Cypress, lo que me ayuda a entender cómo se integran las distintas herramientas dentro de un mismo proceso de calidad.

SonarQube – Análisis estático de código

Plataforma que revisa el código fuente sin ejecutarlo para detectar errores, vulnerabilidades de seguridad, código duplicado y malas prácticas (code smells). Permite definir "quality gates", es decir, umbrales mínimos de calidad que el código debe cumplir antes de integrarse, y se conecta con pipelines de CI/CD. Su Community Edition es gratuita.

Justificación: La agregué para cubrir el análisis estático, un tipo de prueba diferente a las demás porque revisa el código sin ejecutarlo. Me pareció importante porque permite

detectar errores y malas prácticas desde las primeras etapas del desarrollo, lo que se relaciona con el principio de que encontrar un defecto temprano es más barato que corregirlo después.

(TripleTen, 2026)

Análisis de características y funcionalidades

Selenium: Permite controlar el navegador desde código para simular acciones de un usuario. Incluye WebDriver para escribir pruebas, Selenium IDE para grabarlas sin programar y Selenium Grid para ejecutarlas en paralelo en varios navegadores. Su principal ventaja es que es gratuito y flexible en lenguajes y navegadores. Resuelve el problema de las pruebas de regresión manuales, que son repetitivas y lentas, y se integra en la etapa de pruebas funcionales, ejecutándose automáticamente con cada cambio en el código.

Postman: Permite enviar peticiones a APIs, organizarlas en colecciones, manejar distintos entornos y agregar validaciones automáticas a las respuestas. Su interfaz visual lo hace fácil de aprender sin necesidad de programar desde el inicio. Resuelve el problema de probar el backend sin depender de que la interfaz esté terminada, por lo que puede usarse desde etapas tempranas del desarrollo. Con su herramienta Newman, las pruebas se pueden ejecutar de forma automática.

Apache JMeter: Simula muchos usuarios accediendo al mismo tiempo a una aplicación y muestra los resultados en gráficas y reportes, como tiempos de respuesta y cantidad de errores. Es gratuito y permite diseñar las pruebas desde una interfaz gráfica. Resuelve el problema de no saber cuánta carga soporta un sistema antes de fallar, y se usa principalmente antes de salir a producción, en un ambiente similar al real.

Cypress: Ejecuta las pruebas dentro del navegador y muestra paso a paso lo que ocurre, con capturas de cada acción y videos cuando una prueba falla. Espera automáticamente a que los elementos carguen, lo que reduce las pruebas que fallan sin motivo real. Su ventaja es la facilidad de instalación y de depuración, y se integra bien con proyectos de frontend en JavaScript, donde los desarrolladores pueden escribir las pruebas junto con el código.

Allure: Genera reportes visuales a partir de los resultados de otras herramientas, mostrando los pasos de cada prueba, capturas de pantalla, tiempos e historial. Allure TestOps agrega la gestión de pruebas manuales y automatizadas en un solo lugar. Resuelve el problema de tener resultados dispersos o difíciles de leer, y se usa al final de cada ejecución para comunicar los resultados a todo el equipo.

SonarQube: Revisa el código fuente sin ejecutarlo para encontrar errores, vulnerabilidades de seguridad, código duplicado y malas prácticas. Permite definir condiciones mínimas de calidad que el código debe cumplir antes de integrarse. Su ventaja es que detecta los problemas en la etapa más temprana, cuando corregirlos es más barato, ya que se ejecuta cada vez que se escribe o se sube código al repositorio.

Integración en el proceso de desarrollo: Las seis herramientas se

complementan porque cubren etapas distintas: SonarQube revisa el código, Postman valida las APIs, Selenium y Cypress prueban la aplicación desde la interfaz, JMeter evalúa el rendimiento y Allure reúne los resultados. En un proceso de integración continua, todas pueden ejecutarse de forma automática.

(QAlified, 2024)

Herramienta	Facilidad de uso	Costo	Soporte de plataformas	Integración	Comunidad
Selenium	Media-baja: requiere saber programar	Gratuito	Windows, macOS, Linux; Chrome, Firefox, Edge, Safari	Alta: frameworks de pruebas, CI/CD, Allure	Muy grande, la más antigua
Postman	Alta: interfaz visual	Plan gratuito con límites; planes de pago	Windows, macOS, Linux y versión web	Alta: CI/CD con Newman	Muy grande
JMeter	Media: muchos conceptos que aprender	Gratuito	Cualquier sistema con Java	Media-alta: CI/CD por línea de comandos, plugins	Grande
Cypress	Alta para quien conoce JavaScript	Gratuito; Cypress Cloud de pago	Windows, macOS, Linux; Chrome, Edge, Firefox	Alta: CI/CD, Allure	Grande y en crecimiento
Allure	Media: configurar es técnico, leer reportes es fácil	Report gratuito; TestOps de pago	Multiplataforma	Muy alta: se conecta con casi todos los frameworks	Mediana

SonarQube	Media: requiere instalar un servidor	Community gratuita; ediciones de pago	Servidor en Windows, Linux, macOS o Docker; analiza más de 30 lenguajes	Alta: CI/CD, GitHub, GitLab, editores de código	Alta: CI/CD, GitHub, GitLab, editores de código
-----------	--------------------------------------	---------------------------------------	---	---	---

Ventajas y desventajas

Selenium: Su mayor ventaja es la flexibilidad, funciona con varios lenguajes y navegadores, y al ser el más antiguo tiene muchísima documentación; y su desventaja, requiere más código y configuración que otras herramientas, no trae reportes propios y las pruebas pueden volverse inestables si no se manejan bien los tiempos de espera.

Postman: Es muy fácil de empezar a usar y permite probar APIs sin programar. Sus desventajas son que el plan gratuito limita el trabajo en equipo, que requiere una cuenta y guarda información en la nube, y que no está pensado para pruebas de carga a gran escala.

JMeter: Es gratuito y muy completo para pruebas de rendimiento. Sin embargo, su interfaz es poco moderna, consume muchos recursos cuando se simulan muchos usuarios desde un solo equipo y no ejecuta JavaScript como un navegador real, por lo que no mide exactamente la experiencia del usuario final.

Cypress: Destaca por su facilidad de instalación, su velocidad y lo fácil que es encontrar la causa de un fallo. Sus desventajas es que solo trabaja con JavaScript y TypeScript, tiene restricciones con múltiples pestañas y no sirve para aplicaciones móviles nativas.

Allure: Hace que los resultados sean claros y fáciles de compartir, incluso con personas no técnicas, su desventaja, no ejecuta pruebas por sí mismo, requiere configurarse para cada framework y la versión con gestión de pruebas (TestOps) tiene costo.

SonarQube: Detecta problemas en la etapa más temprana y ayuda a mantener un estándar de calidad en el código. Sus desventajas son que necesita un servidor y recursos para funcionar, puede mostrar falsos positivos que hay que revisar manualmente, y la versión gratuita tiene funciones limitadas para analizar ramas y solicitudes de cambio.

Caso de estudio Boeing Starliner (2019)

En diciembre de 2019, Boeing lanzó la cápsula Starliner en un vuelo de prueba sin tripulación hacia la Estación Espacial Internacional. Poco después del lanzamiento, el reloj interno de la misión (Mission Elapsed Timer) de la cápsula tomó la hora del cohete en un momento equivocado y quedó desfasado unas 11 horas. Como la cápsula "creía" estar en otra fase de la misión, encendió sus propulsores cuando no debía, gastó demasiado combustible y no pudo llegar a la Estación. Durante el vuelo se encontró además un segundo error de software, en la secuencia de separación del módulo de servicio, que pudo

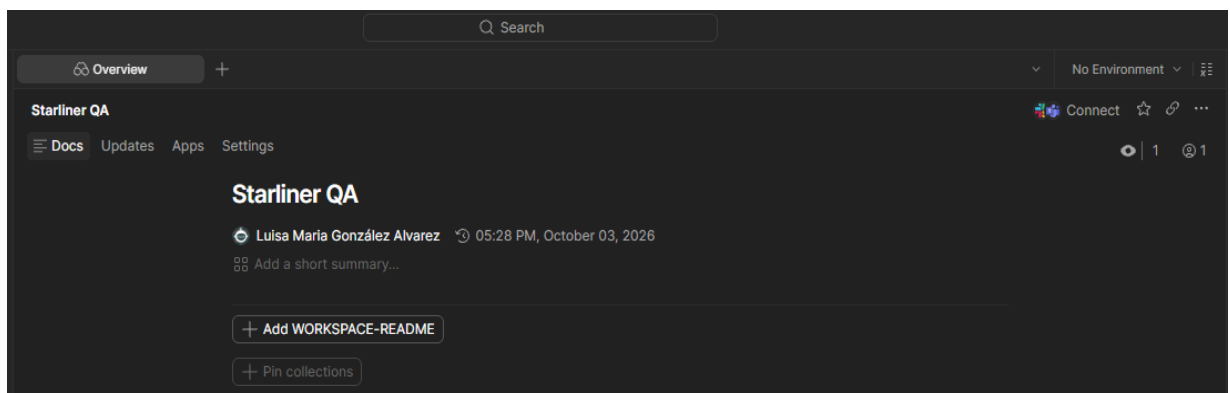
haber causado un choque entre los módulos; se corrigió desde tierra antes de que ocurriera. La cápsula aterrizó a salvo dos días después.

La revisión independiente de NASA concluyó que el problema de fondo fue de pruebas: Boeing no ejecutó una prueba integrada completa de principio a fin con el cohete, sino pruebas por partes, y por eso los errores no aparecieron antes del vuelo. NASA lo clasificó como un incidente grave ("high visibility close call") y Boeing tuvo que repetir la misión en 2022.

(Marín, 2019) o "Según Marín (2019), ..."

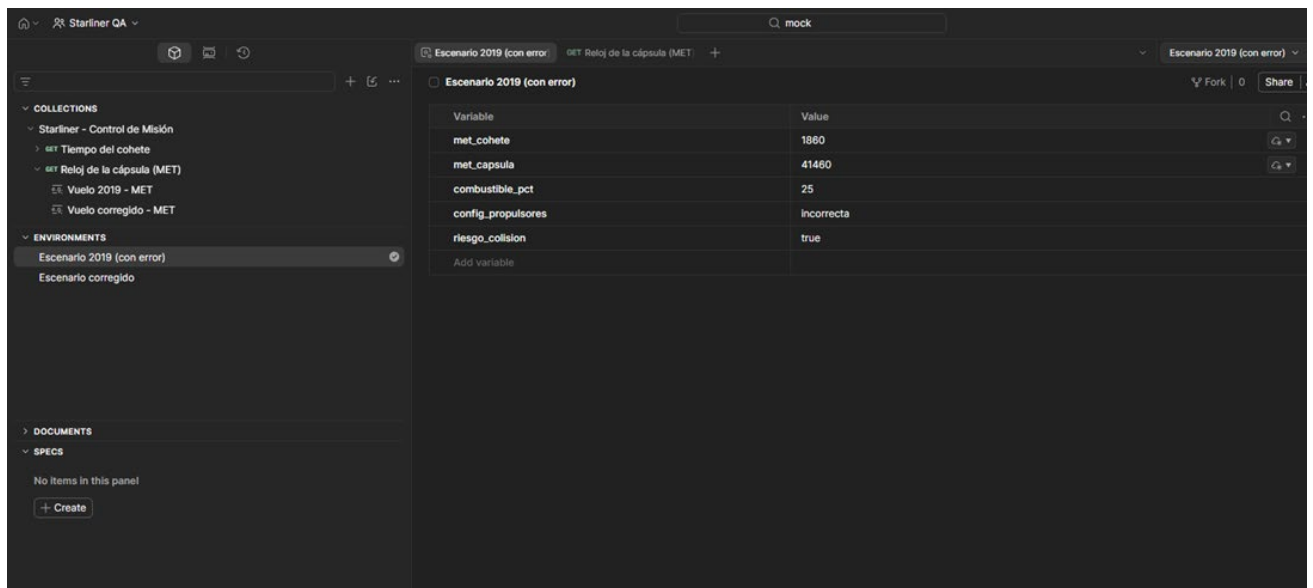
Se utilizó la versión web de Postman con una cuenta gratuita. Se creó un espacio de trabajo llamado "Starliner QA" para organizar todos los elementos de la práctica.

Capturas del proceso y uso de la herramienta Postman

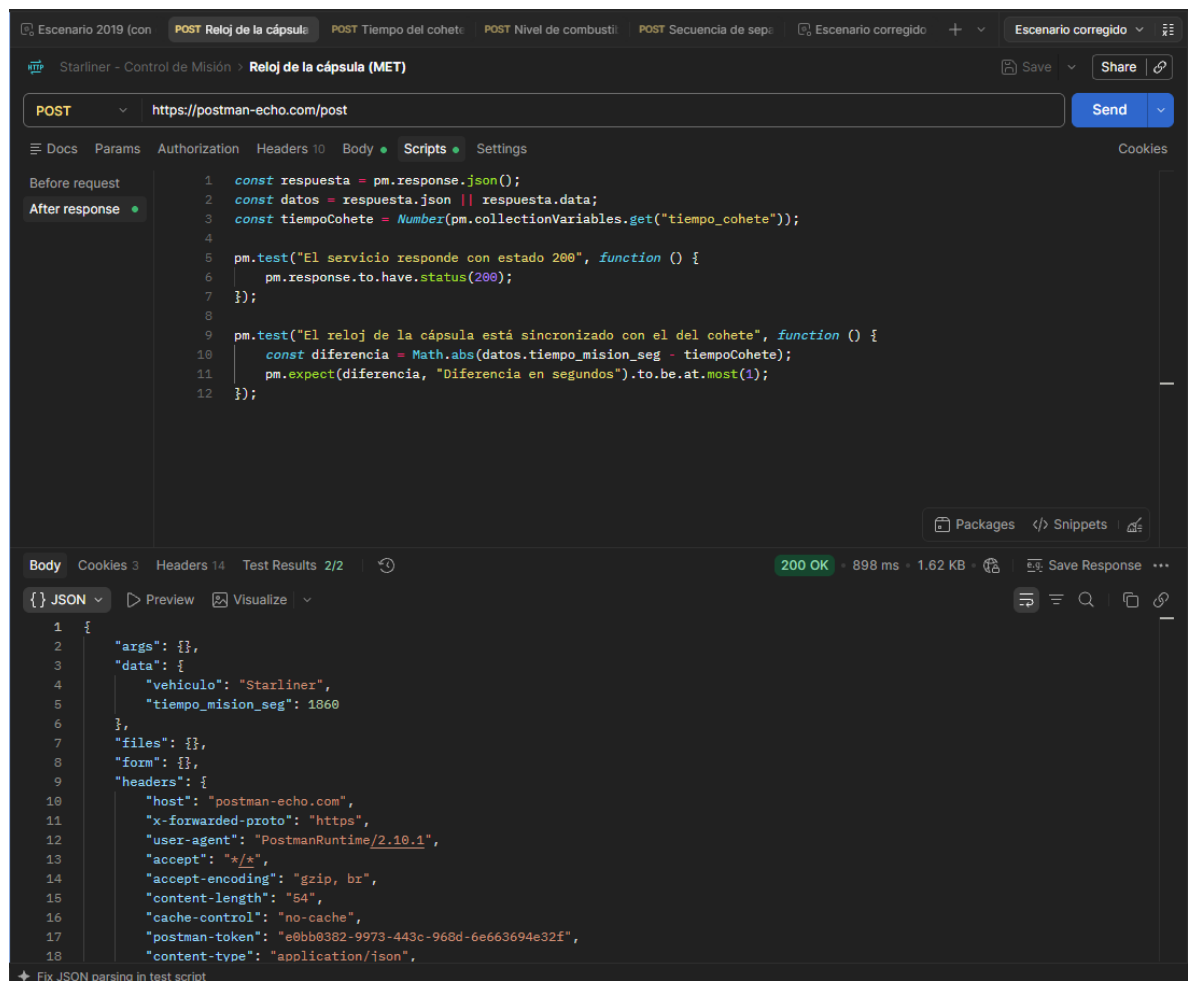
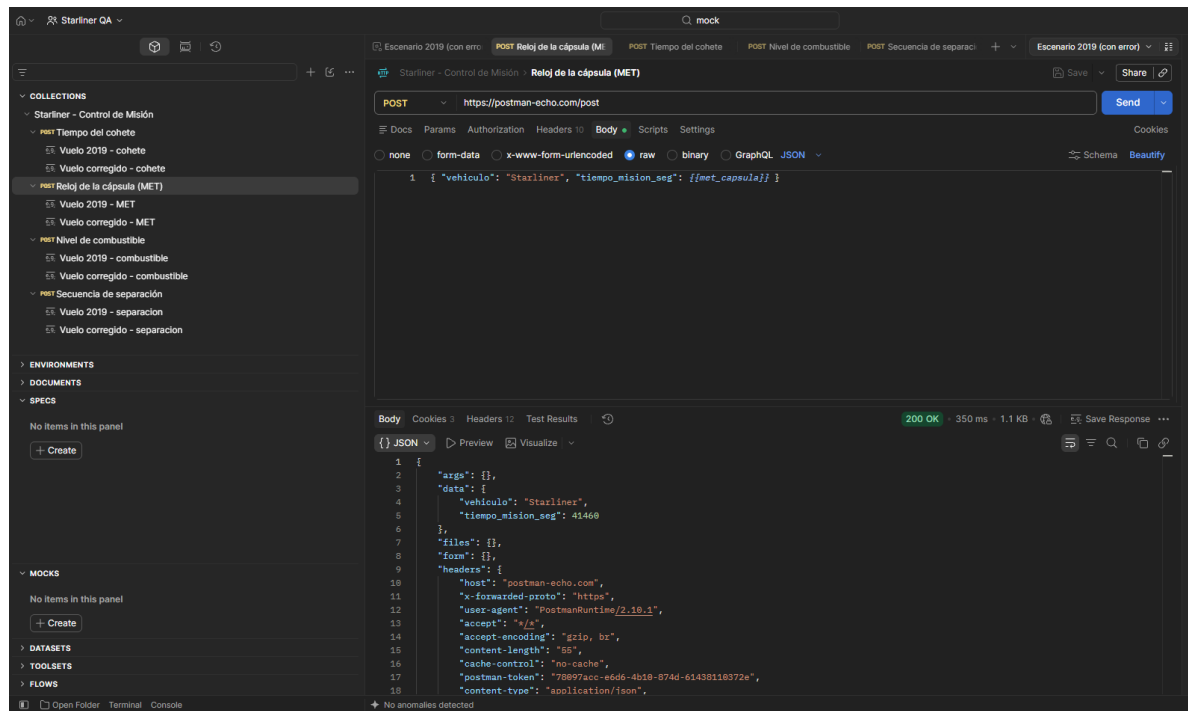


Se creó la colección "Starliner - Control de Misión" con las cuatro peticiones de tipo POST dirigidas a Postman Echo. El cuerpo de cada petición contiene los datos de

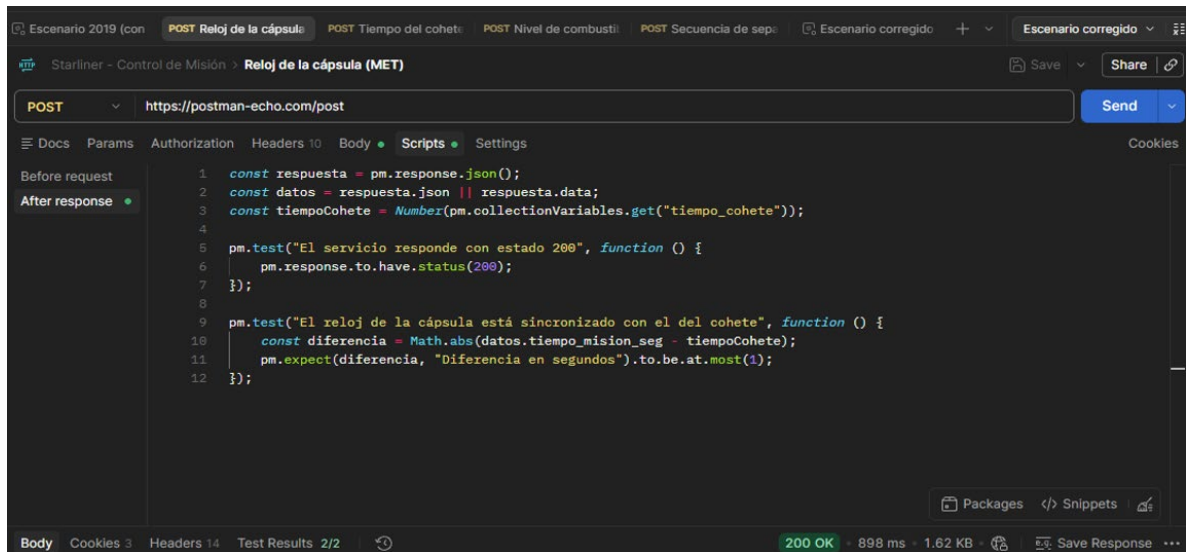
telemetría en formato JSON, con variables que toman su valor del entorno seleccionado, lo que permite ejecutar exactamente las mismas peticiones con los datos de cualquiera de los dos escenarios.



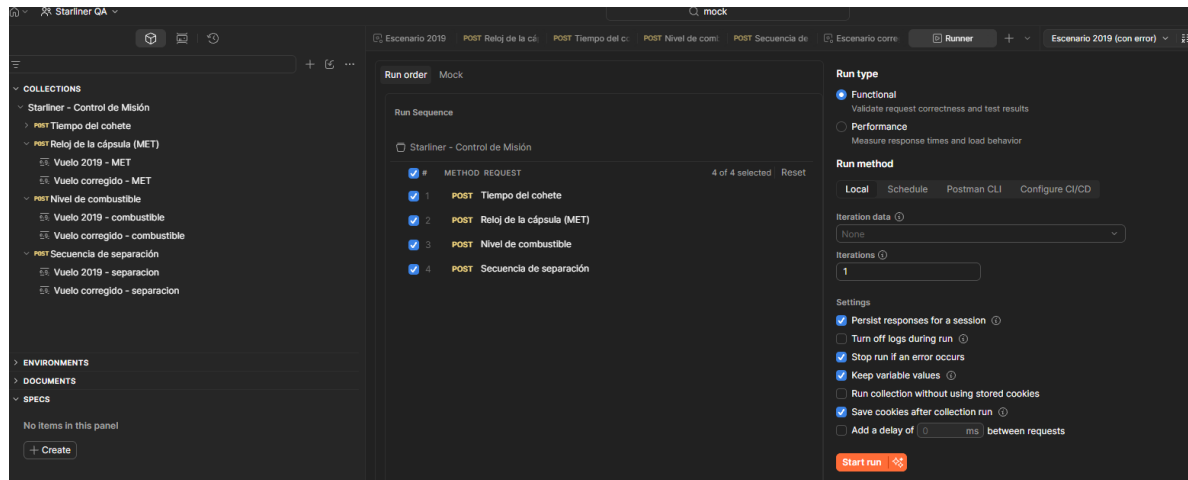
Antes de escribir las pruebas se verificó que el reemplazo de variables funcionara: con el escenario de 2019, el reloj de la cápsula reportó 41.460 segundos, mientras que con el escenario corregido reportó 1.860 segundos, igual que el cohete.



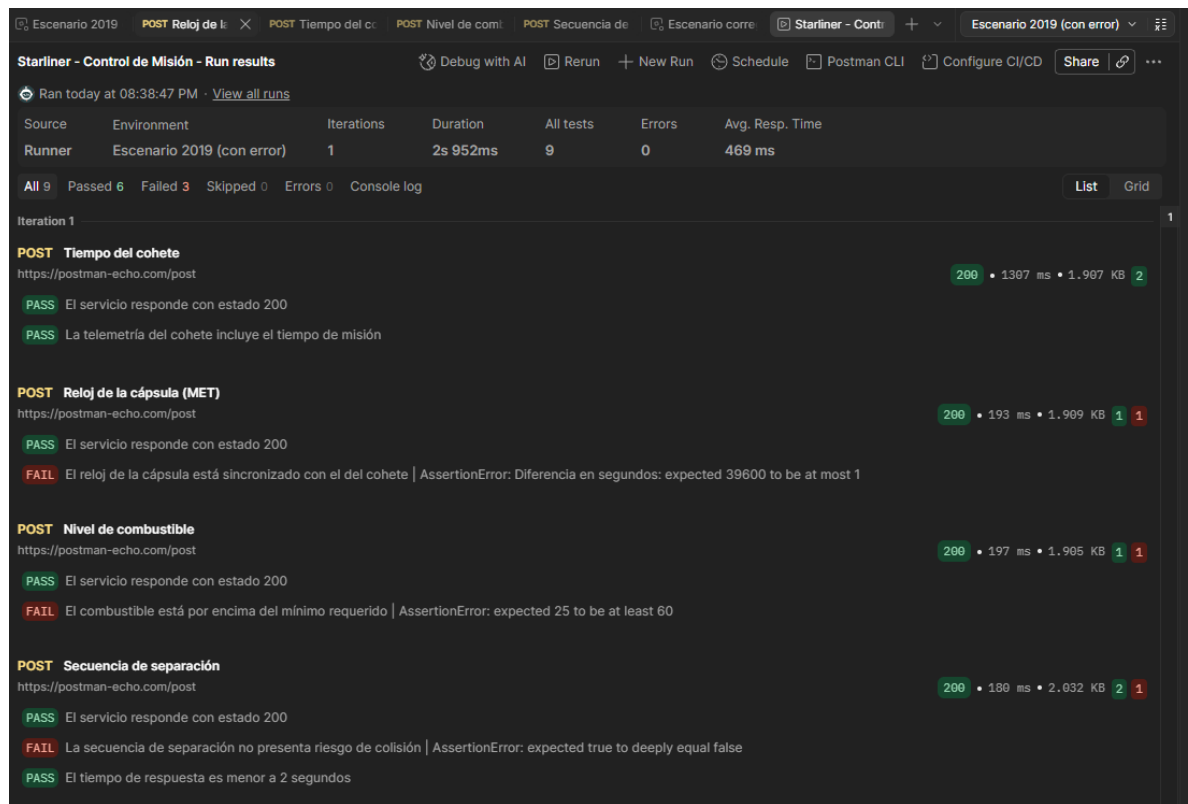
En la sección de scripts de cada petición se escribieron pruebas automáticas en JavaScript, para un total de nueve validaciones: el código de estado de las cuatro respuestas, la presencia del tiempo de misión del cohete, la sincronización entre los relojes del cohete y la cápsula, el nivel de combustible frente al mínimo requerido, la ausencia de riesgo de colisión en la separación y el tiempo de respuesta del servicio.

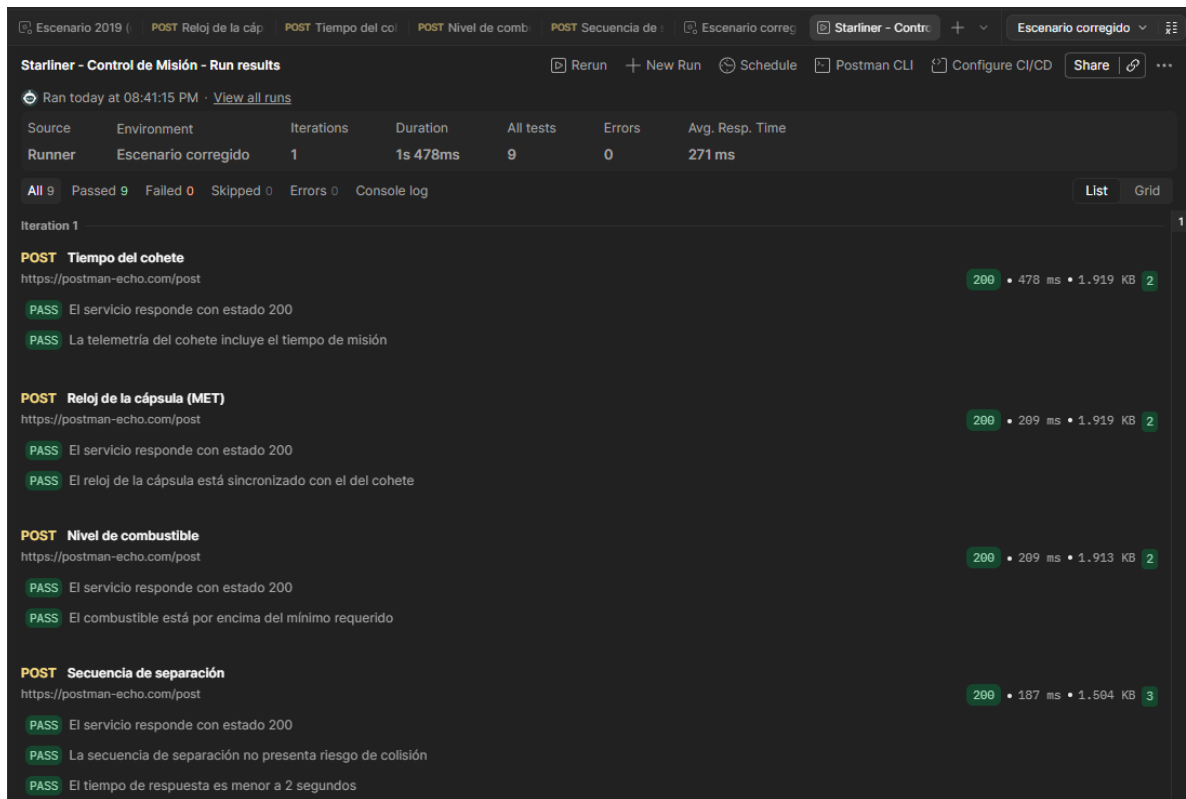


Las pruebas se ejecutaron con el Collection Runner de Postman, que corre todas las peticiones de la colección en orden y consolida los resultados. Se realizaron dos ejecuciones con la misma configuración, una por cada entorno.



Las pruebas se ejecutaron con el Collection Runner de Postman, que corre todas las peticiones de la colección en orden y consolida los resultados. Se realizaron dos ejecuciones con la misma configuración, una por cada entorno.





Como no es posible probar el software real de la cápsula, se diseñó una simulación didáctica en Postman que representa la comunicación entre los sistemas del vuelo y el centro de control. Cada petición simula un mensaje de telemetría enviado al control de misión, y las pruebas automatizadas verifican si los datos recibidos son correctos.

Para recibir las peticiones se utilizó Postman Echo, un servicio de prueba oficial de Postman que devuelve los datos enviados. Los valores de cada escenario se almacenaron en dos entornos: Escenario 2019 (con error), con los datos que reproducen las fallas del vuelo real, y Escenario corregido, con valores correctos. El desfase del reloj corresponde al caso real; los valores de combustible son ilustrativos.

Conclusiones:

- La comparación realizada permitió identificar que las herramientas de calidad de software no compiten entre sí, sino que cubren etapas distintas del desarrollo: SonarQube actúa sobre el código, Postman sobre las APIs, Selenium y Cypress sobre la interfaz, JMeter sobre el rendimiento y Allure sobre la comunicación de resultados. La elección depende del tipo de proyecto, el presupuesto, el lenguaje y la etapa que se quiere cubrir.
- La aplicación de Postman al caso Boeing Starliner demostró que pruebas automatizadas relativamente sencillas pueden detectar errores graves. Con los datos del vuelo de 2019, tres de las nueve pruebas fallaron e identificaron los dos defectos de software del caso real; con los datos corregidos, todas fueron aprobadas.

Aprendizajes:

- Se aprendió a usar Postman para crear colecciones, configurar entornos con variables, escribir pruebas automáticas en JavaScript y ejecutarlas de forma conjunta con el Collection Runner.
- Se comprendió la diferencia entre una prueba que verifica que un servicio responde y una prueba que verifica que la información sea correcta y coherente con la de otros sistemas.

- Se aprendió que, ante un resultado inesperado, es necesario analizar si el problema está en el sistema probado o en la configuración de la propia herramienta, y que en ocasiones la mejor decisión es cambiar a una alternativa más simple.

Recomendaciones:

- Incorporar pruebas de integración de extremo a extremo en sistemas críticos, ejecutándolas antes de cada entrega y no solo pruebas por componentes.
- Utilizar entornos para separar los datos de prueba, de manera que el mismo conjunto de pruebas pueda ejecutarse en distintos escenarios.
- Para quienes comienzan con Postman, iniciar con la aplicación de escritorio y con servicios simples como Postman Echo antes de usar funciones más avanzadas como los Mock Servers.

Referencias bibliográficas:

- “Se utilizó inteligencia artificial como apoyo para el uso de la herramienta Postman, guiando el paso a paso”
- TripleTen. (2026, 27 de febrero). Descubre las 10 mejores herramientas de prueba de software para el Software Testing en 2026.
<https://tripleten.mx/blog/las-10-mejores-herramientas-de-prueba-de-software-para-testing/>
- QAlified. (2024, 29 de mayo). Las 10 mejores herramientas de testing de software del 2024. <https://qalified.com/es/blog/software-qa-testing-herramientas/>

- Marín, D. (2019, 20 de diciembre). Problemas en la primera misión de la nave Starliner de Boeing. Eureka.

<https://danielmarin.naukas.com/2019/12/20/problemas-en-la-primer-mision-de-la-nave-starliner-de-boeing/>